



Université Ibn Tofaïl — Faculté des Sciences, Kénitra
Master M1 — Big Data, Intelligence Artificielle et Applications Avancées
Module : Python Avancé | Mini-projet

Détection de fraude bancaire

Application web avec Flask et Machine Learning

Rapport de projet

Réalisé par : Bilal Jellaoui

Encadré par : Pr. Hatim Derrouz

Année universitaire 2025–2026

Résumé

La fraude par carte bancaire est un problème de classification binaire extrêmement déséquilibré : dans le jeu de données utilisé (Kaggle, 284 807 transactions), seules 492 transactions sont frauduleuses, soit 0,173 %. Ce rapport présente la conception d’une application web complète, développée avec Flask, qui prédit en temps réel la probabilité de fraude d’une transaction et lui associe un niveau de risque.

Quatre modèles sont comparés : régression logistique, forêt aléatoire, XGBoost et Isolation Forest. Le prétraitement comprend une séparation stratifiée 80/20, une normalisation par `StandardScaler` et un rééchantillonnage SMOTE appliqué uniquement à l’entraînement. Les hyperparamètres sont explorés par `RandomizedSearchCV` avec une validation croisée stratifiée à 5 plis, en maximisant le F1-score.

XGBoost est retenu : sur le jeu de test de 56 962 transactions, il obtient un ROC-AUC de 0,9820, un F1-score de 0,8705, détecte 84 fraudes sur 98 et ne génère que 11 faux positifs. Le modèle est servi par une API REST, avec journalisation des prédictions dans SQLite et un tableau de bord d’analyse des faux positifs et faux négatifs.

Mots-clés : détection de fraude, données déséquilibrées, SMOTE, XGBoost, Flask, API REST, ROC-AUC.

Abstract

Credit-card fraud detection is a highly imbalanced binary classification problem: in the Kaggle dataset used here, only 492 of 284,807 transactions are fraudulent (0.173%). This report describes a complete Flask web application that predicts the fraud probability of a transaction in real time and assigns a risk level to it.

Four models are compared (logistic regression, random forest, XGBoost and Isolation Forest) using a stratified 80/20 split, standardisation, SMOTE applied to the training set only, and cross-validated randomized hyperparameter search. XGBoost achieves the best trade-off on the 56,962-transaction test set: ROC-AUC 0.9820, F1-score 0.8705, 84 of 98 frauds detected and only 11 false positives. The model is served through a REST API with SQLite logging and an admin dashboard for error analysis.

Keywords: fraud detection, imbalanced data, SMOTE, XGBoost, Flask, REST API, ROC-AUC.

Table des matières

Résumé	i
Abstract	i
1 Introduction	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Objectifs	1
1.4 Organisation du rapport	1
2 État de l’art et travaux connexes	2
2.1 Détection de fraude par apprentissage supervisé	2
2.2 Traitement du déséquilibre des classes	2
2.3 Détection d’anomalies	2
2.4 Évaluation et optimisation	2
3 Données et analyse exploratoire	3
3.1 Présentation du jeu de données	3
3.2 Déséquilibre des classes	3
3.3 Montants et distribution temporelle	3
3.4 Variables discriminantes	4
4 Méthodologie	5
4.1 Chaîne de traitement	5
4.2 Prétraitement	6
4.3 Rééchantillonnage	6
4.4 Modèles comparés	6
4.5 Métriques d’évaluation	7
5 Architecture et implémentation de l’application	7
5.1 Architecture logicielle	7
5.2 Flux de prédiction	8
5.3 API REST	9

5.4	Persistance et traçabilité	9
5.5	Interface utilisateur	9
6	Résultats et discussion	10
6.1	Comparaison des modèles	10
6.2	Analyse	12
6.3	Seuil de décision	12
7	Limites et perspectives	12
7.1	Limites	12
7.2	Perspectives	13
8	Conclusion et bilan	13
	Références	14
A	Exemple d'appel de l'API	14
B	Installation et exécution	15

Table des figures

1	Distribution des classes (284 315 transactions légitimes contre 492 fraudes). . . .	3
2	Distribution du montant (échelle logarithmique) et boîtes à moustaches par classe.	4
3	Distribution temporelle des transactions (densité par heure depuis le début du jeu de données).	4
4	Corrélation de Pearson de chaque variable avec la variable cible Class	5
5	Écart absolu entre les moyennes des fraudes et des transactions légitimes, par variable.	5
6	Pipeline d'apprentissage.	6
7	Flux de traitement d'une prédiction.	8
8	Matrices de confusion des quatre modèles.	11
9	Courbes ROC (gauche) et précision-rappel (droite) des quatre modèles.	11

Liste des tableaux

1	Caractéristiques du jeu de données et découpage.	3
2	Modèles et espaces de recherche d'hyperparamètres.	7
3	Composants de l'application.	8
4	Endpoints exposés par l'application.	9
5	Schéma de la table predictions	9
6	Performances sur le jeu de test (seuil de décision 0,5).	10
7	Matrices de confusion sur le jeu de test (98 fraudes, 56 864 transactions légitimes).	10
8	Bilan de conformité au cahier des charges.	13

1. Introduction

1.1 Contexte

Avec la généralisation du paiement par carte et du commerce en ligne, la fraude bancaire génère chaque année des pertes considérables pour les établissements financiers et leurs clients. Les règles manuelles ne suffisent plus : les schémas de fraude évoluent et le volume de transactions à surveiller est trop important pour un contrôle humain systématique. L'apprentissage automatique permet de scorer chaque transaction en quelques millisecondes et de concentrer l'attention des analystes sur les cas les plus suspects.

1.2 Problématique

Détecter la fraude est difficile pour deux raisons. D'abord, le *déséquilibre extrême* des classes : les fraudes représentent moins de 0,2 % des transactions, si bien qu'un modèle qui prédit systématiquement « légitime » obtient plus de 99,8 % d'accuracy sans détecter aucune fraude. Ensuite, le *coût asymétrique* des erreurs : une fraude manquée coûte de l'argent, mais un faux positif bloque un client légitime et dégrade sa confiance. Le projet doit donc être évalué avec des métriques adaptées (précision, rappel, F1-score, ROC-AUC, courbe précision-rappel) et non avec l'accuracy seule.

1.3 Objectifs

Ce projet, réalisé dans le cadre du module *Python Avancé*, vise à :

- parcourir le cycle complet d'un projet de Machine Learning : collecte, analyse, prétraitement, entraînement, évaluation et déploiement ;
- traiter un jeu de données très déséquilibré (rééchantillonnage SMOTE, pondération des classes) ;
- comparer plusieurs algorithmes supervisés et non supervisé avec des métriques adaptées ;
- exposer le meilleur modèle via une API REST Flask et une interface web utilisable par un agent bancaire.

1.4 Organisation du rapport

La section 2 situe le projet par rapport aux approches usuelles. Les sections 3 et 4 décrivent les données et la méthodologie, la section 5 l'architecture de l'application, et la section 6 les résultats. Les limites, les perspectives et le bilan du projet concluent le document.

2. État de l’art et travaux connexes

2.1 Détection de fraude par apprentissage supervisé

Le jeu de données utilisé provient d’une collaboration entre Worldline et l’Université libre de Bruxelles ; il est devenu une référence pour étudier la détection de fraude par carte [1]. Les travaux de ce groupe soulignent des difficultés propres au domaine : déséquilibre extrême, évolution des comportements dans le temps, et nécessité d’évaluer les modèles avec des métriques qui reflètent le coût réel des erreurs [2]. Parmi les classifieurs usuels, les méthodes d’ensemble à base d’arbres, comme les forêts aléatoires [4] et le *gradient boosting* [5], donnent généralement de bons résultats sur les données tabulaires de ce type.

2.2 Traitement du déséquilibre des classes

Trois familles de solutions sont couramment employées : le *sur-échantillonnage* de la classe minoritaire, dont SMOTE [3] génère des exemples synthétiques par interpolation entre voisins, le *sous-échantillonnage* de la classe majoritaire (par exemple NearMiss), et la *pondération des classes* dans la fonction de coût. Le rééchantillonnage ne doit être appliqué qu’aux données d’entraînement : l’appliquer avant la séparation introduirait une fuite d’information vers le jeu de test. La bibliothèque *imbalanced-learn* [10] fournit ces méthodes.

2.3 Détection d’anomalies

Une alternative consiste à ne pas utiliser les étiquettes et à traiter la fraude comme une anomalie. Isolation Forest [6] isole les observations atypiques par des partitions aléatoires ; il est entraîné ici sur les seules transactions légitimes. Cette approche est séduisante lorsque les étiquettes sont rares, mais elle est en général moins performante qu’un modèle supervisé lorsque des fraudes étiquetées sont disponibles, ce que confirment nos résultats.

2.4 Évaluation et optimisation

Sur des données très déséquilibrées, la courbe précision-rappel est plus informative que la courbe ROC [8] : le ROC-AUC peut rester élevé alors que la précision réelle est faible. Nous rapportons donc les deux. Pour les hyperparamètres, la recherche aléatoire est une alternative efficace à la recherche exhaustive [7], et c’est l’approche retenue ici. Enfin, les valeurs SHAP [9] permettent d’expliquer une prédiction individuelle ; elles sont envisagées comme extension (section 7).

3. Données et analyse exploratoire

3.1 Présentation du jeu de données

Le jeu de données *Credit Card Fraud Detection* regroupe des transactions par carte réalisées par des titulaires européens sur deux jours. Pour des raisons de confidentialité, les variables V_1 à V_{28} sont le résultat d'une analyse en composantes principales (ACP) et ne sont pas interprétables ; seules **Time** (secondes écoulées depuis la première transaction) et **Amount** (montant) sont fournies en clair.

Table 1. Caractéristiques du jeu de données et découpage.

Caractéristique	Valeur
Transactions (total)	284 807
Transactions légitimes	284 315
Transactions frauduleuses	492 (0,173 %)
Variables	31 (Time , Amount , V_1 – V_{28} , Class)
Valeurs manquantes	aucune
Entraînement (80 %, stratifié)	227 845
Test (20 %, stratifié)	56 962 (dont 98 fraudes)

3.2 Déséquilibre des classes

La figure 1 confirme le déséquilibre extrême : les fraudes sont à peine visibles dans le diagramme en barres. Ce constat conditionne tout le reste de la démarche, du choix des métriques au rééchantillonnage.

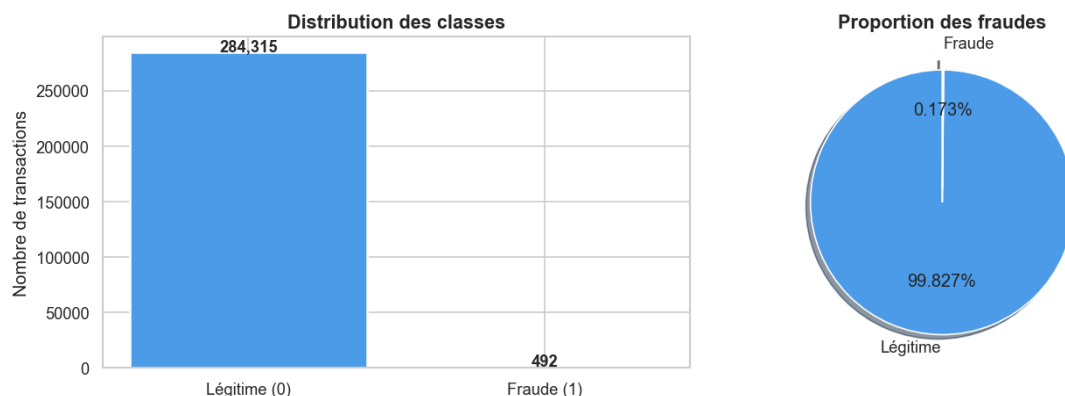


Figure 1. Distribution des classes (284 315 transactions légitimes contre 492 fraudes).

3.3 Montants et distribution temporelle

La distribution des montants est très asymétrique (échelle logarithmique) : la grande majorité des transactions porte sur de faibles montants. La boîte à moustaches montre que les montants frauduleux sont légèrement décalés vers le haut par rapport aux montants légitimes. Sur l'axe

temporel, les transactions légitimes suivent un rythme jour/nuit régulier, tandis que les fraudes se répartissent de façon plus irrégulière, avec des pics ponctuels, y compris pendant les heures creuses où la proportion de fraudes parmi les transactions est la plus forte.

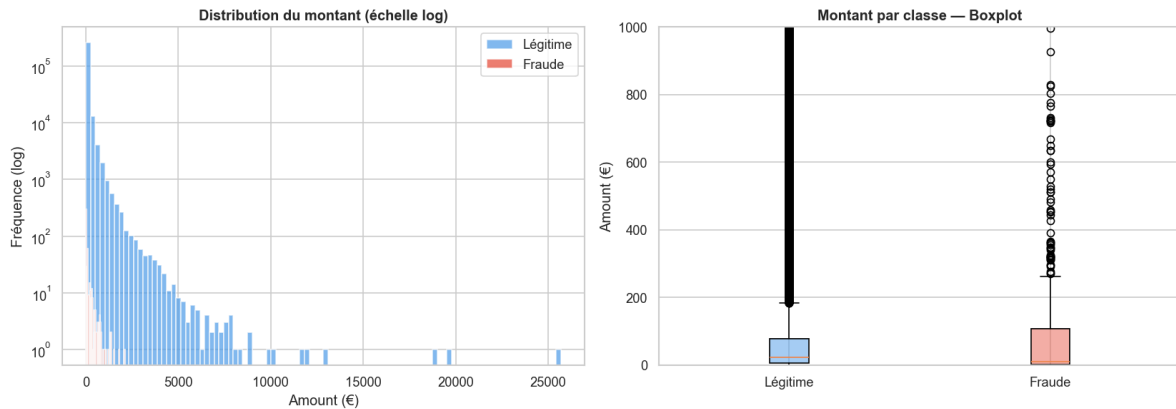


Figure 2. Distribution du montant (échelle logarithmique) et boîtes à moustaches par classe.

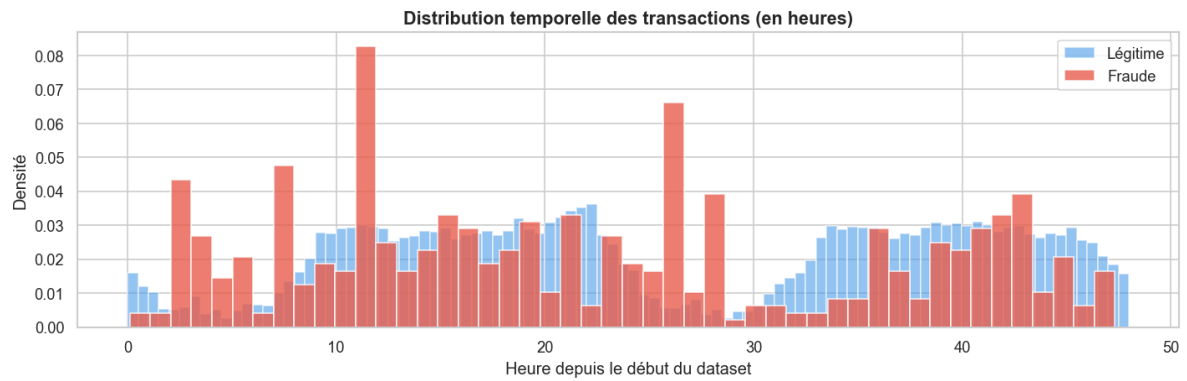


Figure 3. Distribution temporelle des transactions (densité par heure depuis le début du jeu de données).

3.4 Variables discriminantes

Deux analyses complémentaires identifient les variables les plus informatives. La figure 4 présente la corrélation de Pearson de chaque variable avec la cible : V_{17} , V_{14} , V_{12} et V_{10} sont les plus fortement corrélées négativement, V_{11} et V_4 les plus corrélées positivement. La figure 5 classe les variables par écart absolu entre la moyenne des fraudes et celle des transactions légitimes ; V_3 , V_{14} , V_{17} et V_{12} y dominent. Les deux approches s'accordent sur le rôle de V_{14} , V_{17} et V_{12} .

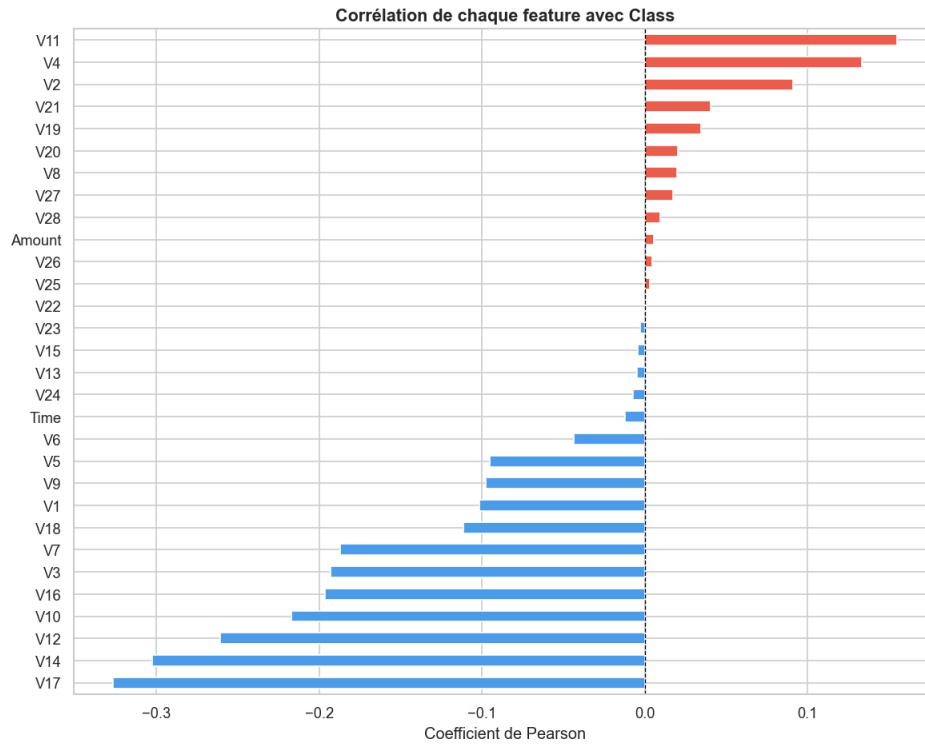


Figure 4. Corrélation de Pearson de chaque variable avec la variable cible `Class`.

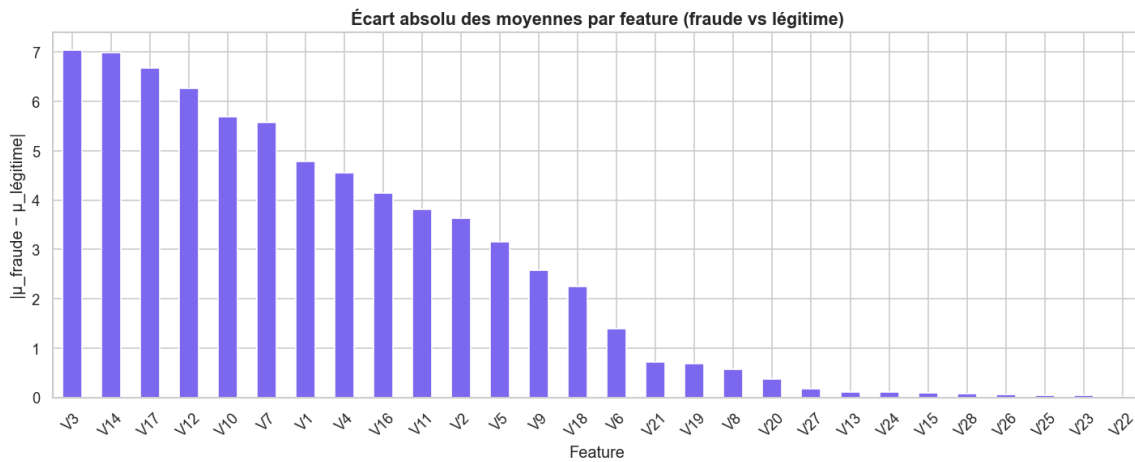


Figure 5. Écart absolu entre les moyennes des fraudes et des transactions légitimes, par variable.

4. Méthodologie

4.1 Chaîne de traitement

Le pipeline d'apprentissage est implémenté dans le dossier `ml/` (`preprocess.py`, `train.py`, `evaluate.py`). La figure 6 en résume les étapes.

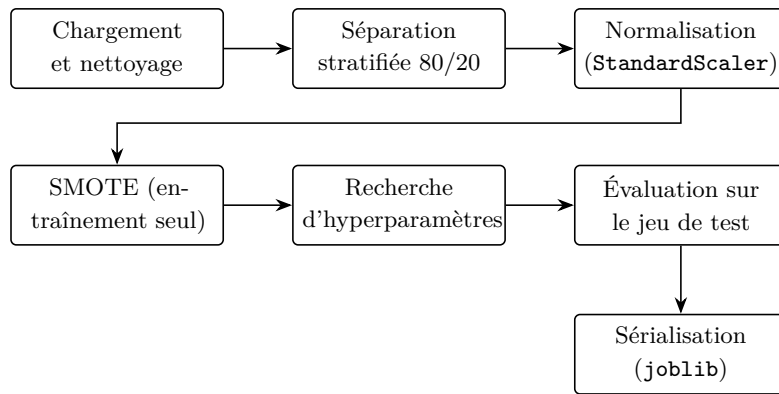


Figure 6. Pipeline d'apprentissage.

4.2 Prétraitement

- **Valeurs manquantes** : aucune n'est présente, mais le traitement est prévu dans le pipeline.
- **Séparation** : découpage stratifié 80/20 avec graine fixée (`random_state=42`), de sorte que la proportion de fraudes est conservée dans l'entraînement (394 fraudes) et le test (98 fraudes).
- **Normalisation** : `Amount` et `Time` sont standardisés par `StandardScaler` ; les variables V_1 à V_{28} , issues d'une ACP, ne sont pas retraitées. Le *scaler* est ajusté sur l'entraînement puis sauvegardé pour être réutilisé à l'inférence.

4.3 Rééchantillonnage

SMOTE est appliqué *uniquement* sur l'ensemble d'entraînement, après la séparation et la normalisation, avec un ratio fraudes/légitimes de 10 % et $k = 5$ voisins. Le jeu de test reste intact et conserve la distribution réelle. Le code propose aussi NearMiss (sous-échantillonnage) comme alternative, non retenue dans les résultats présentés.

Listing 1. Rééchantillonnage SMOTE (extrait de `ml/preprocess.py`).

```
smote = SMOTE(
    sampling_strategy=0.1,  # fraudes = 10 % des legitimes
    random_state=42,
    k_neighbors=5
)
X_res, y_res = smote.fit_resample(X_train, y_train)
```

4.4 Modèles comparés

Quatre modèles sont entraînés. Les trois modèles supervisés sont optimisés par `RandomizedSearchCV` (7 combinaisons tirées aléatoirement, validation croisée stratifiée à 5 plis, critère : F1-score) ; l'Isolation Forest, non supervisé, est entraîné sans recherche d'hyperparamètres sur les seules transactions légitimes.

Table 2. Modèles et espaces de recherche d’hyperparamètres.

Modèle	Configuration / espace de recherche
Régression logistique	<code>class_weight='balanced'</code> ; $C \in \{0,001; 0,01; 0,1; 1; 10; 100\}$; pénalité ℓ_2
Random Forest	<code>class_weight='balanced'</code> ; <code>n_estimators</code> $\in \{50, 100, 150\}$; <code>max_depth</code> $\in \{\text{None}, 10, 20\}$; <code>min_samples_split</code> $\in \{2, 5, 10\}$; <code>min_samples_leaf</code> $\in \{1, 2, 4\}$; <code>max_features</code> $\in \{\text{sqrt}, \text{log2}\}$
XGBoost	<code>n_estimators</code> $\in \{50, 100, 150\}$; <code>max_depth</code> $\in \{3, 5, 7, 9\}$; <code>learning_rate</code> $\in \{0,01; 0,05; 0,1; 0,2\}$; <code>subsample</code> et <code>colsample_bytree</code> $\in \{0,7; 0,8; 1,0\}$; <code>scale_pos_weight</code> $\in \{1, 10, 50, 100\}$
Isolation Forest	<code>n_estimators=200</code> , <code>contamination=0.001</code> , entraîné sur la classe légitime uniquement

4.5 Métriques d’évaluation

Soient TP , FP , FN et TN les effectifs de la matrice de confusion. Les métriques utilisées sont :

$$\text{Précision} = \frac{TP}{TP + FP}, \quad \text{Rappel} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}.$$

La précision mesure la proportion d’alertes justifiées, le rappel la proportion de fraudes détectées. S’y ajoutent le ROC-AUC, l’*Average Precision* (aire sous la courbe précision-rappel) et l’accuracy, cette dernière n’étant donnée qu’à titre indicatif en raison du déséquilibre. Toutes les métriques sont calculées sur le jeu de test avec le seuil de décision de 0,5.

5. Architecture et implémentation de l’application

5.1 Architecture logicielle

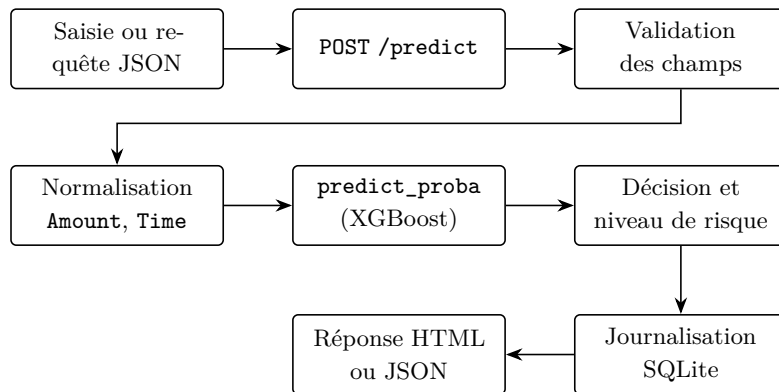
L’application sépare le pipeline d’apprentissage (`ml/`) du service web (`app/`), ce qui permet de faire évoluer l’un sans toucher à l’autre. Elle suit le patron *Application Factory* avec un *Blueprint* Flask. Le modèle est chargé une seule fois au démarrage via la classe `FraudDetector`, instanciée globalement.

Table 3. Composants de l'application.

Composant	Fichier	Rôle
Application Factory	app/__init__.py	Création de l'application, base SQLite, chargement du modèle
Routes	app/routes.py	Endpoints REST et modèle SQLAlchemy <code>Prediction</code>
Wrapper ML	app/models/fraud_detector.py	Chargement du modèle, prétraitement, prédiction
Prétraitement	ml/preprocess.py	Chargement, séparation, normalisation, SMOTE
Entraînement	ml/train.py	Recherche d'hyperparamètres, sauvegarde du modèle
Évaluation	ml/evaluate.py	Métriques, courbes ROC et PR, matrices de confusion
Interface	app/templates/	Formulaire de saisie (<code>index.html</code>) et résultat (<code>result.html</code>)

5.2 Flux de prédiction

Lorsqu'un agent soumet une transaction, la requête traverse les étapes de la figure 7. Les champs **Amount** et **Time** sont obligatoires ; une requête incomplète reçoit une erreur HTTP 400, et l'absence de modèle chargé renvoie une erreur HTTP 503. Les variables non fournies sont initialisées à zéro par le wrapper.

**Figure 7.** Flux de traitement d'une prédiction.

5.3 API REST

Table 4. Endpoints exposés par l'application.

Méthode	Route	Description
GET	/	Formulaire de saisie d'une transaction
POST	/predict	Probabilité de fraude et niveau de risque (JSON ou formulaire)
GET	/predict/<id>	Résultat d'une prédiction précise
GET	/stats	Statistiques du modèle et des prédictions étiquetées
GET	/history	50 dernières prédictions (nombre paramétrable)
GET	/admin	Tableau de bord des faux positifs et faux négatifs

L'annexe A donne un exemple d'appel. La réponse contient la décision (**is_fraud**), la probabilité de fraude, le niveau de risque (**LOW**, **MEDIUM** ou **HIGH**), le nom du modèle et l'identifiant de la prédiction enregistrée.

5.4 Persistance et traçabilité

Chaque prédiction est enregistrée dans une base SQLite via SQLAlchemy (table **predictions**). Le champ optionnel **true_label** permet à un agent de renseigner la vérité terrain ; il alimente les statistiques en production et le tableau de bord administrateur, qui calcule alors les vrais/faux positifs et négatifs observés.

Table 5. Schéma de la table **predictions**.

Colonne	Type	Description
id	entier (clé primaire)	Identifiant de la prédiction
timestamp	date-heure (UTC)	Horodatage
amount	réel	Montant de la transaction
time_seconds	réel	Variable Time
fraud_probability	réel	Probabilité de fraude
is_fraud	booléen	Décision du système
risk_level	texte	LOW , MEDIUM ou HIGH
input_features	texte (JSON)	Variables d'entrée complètes
true_label	entier, optionnel	Vérité terrain renseignée a posteriori

5.5 Interface utilisateur

L'interface est servie par des gabarits Jinja2 : une page de saisie des variables de la transaction et une page de résultat affichant la probabilité et le niveau de risque. Le tableau de bord **/admin** liste les faux positifs et les faux négatifs observés parmi les prédictions étiquetées.

6. Résultats et discussion

6.1 Comparaison des modèles

Le tableau 6 présente les performances des quatre modèles sur le jeu de test (56 962 transactions, dont 98 fraudes), au seuil de décision de 0,5.

Table 6. Performances sur le jeu de test (seuil de décision 0,5).

Modèle	Accuracy	Précision	Rappel	F1	ROC-AUC	AP
XGBoost	0,9996	0,8842	0,8571	0,8705	0,9820	0,879
Random Forest	0,9995	0,8710	0,8265	0,8482	0,9689	0,874
Régression logistique	0,9761	0,0622	0,9184	0,1166	0,9726	0,730
Isolation Forest	0,9975	0,2152	0,1735	0,1921	0,5862	0,039

Table 7. Matrices de confusion sur le jeu de test (98 fraudes, 56 864 transactions légitimes).

Modèle	VN	FP	FN	VP
XGBoost	56 853	11	14	84
Random Forest	56 852	12	17	81
Régression logistique	55 508	1 356	8	90
Isolation Forest	56 802	62	81	17

Matrices de confusion — Tous les modèles

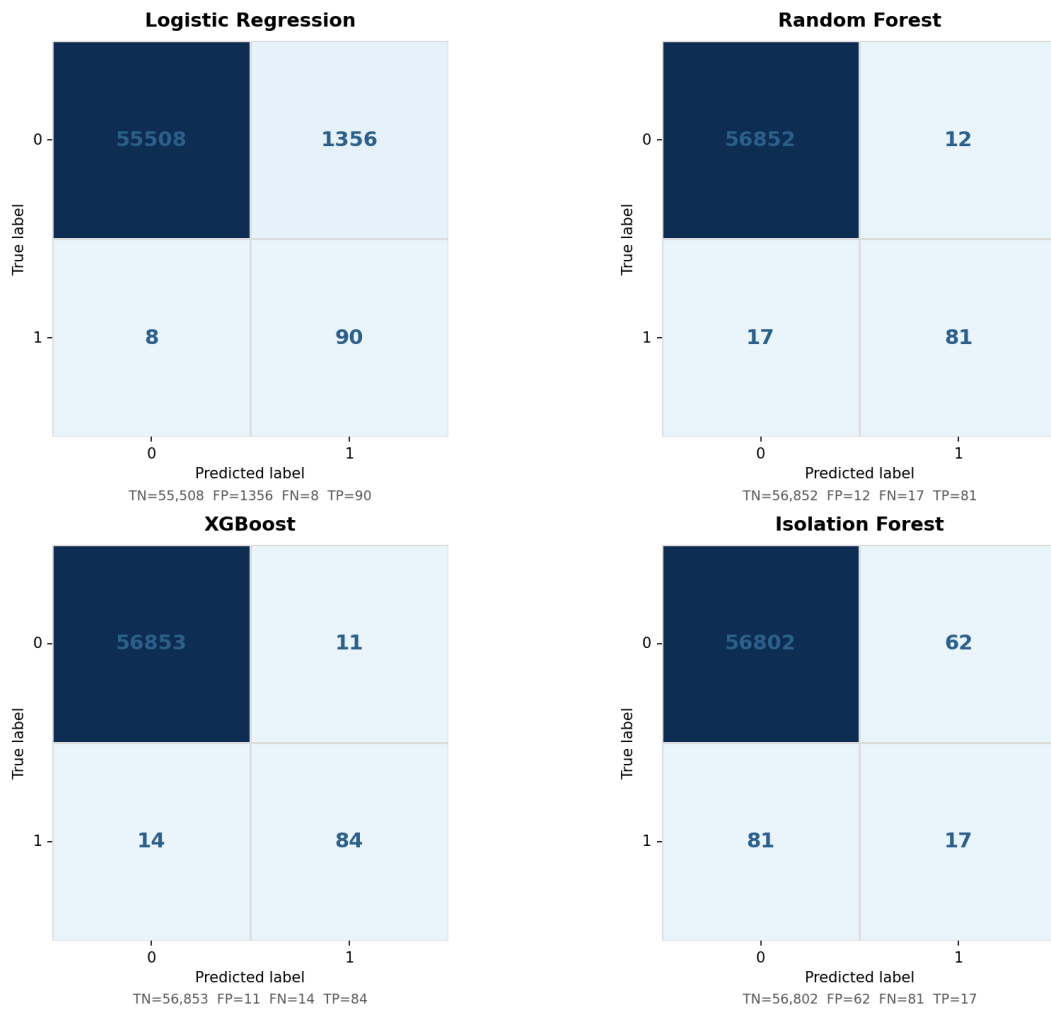


Figure 8. Matrices de confusion des quatre modèles.

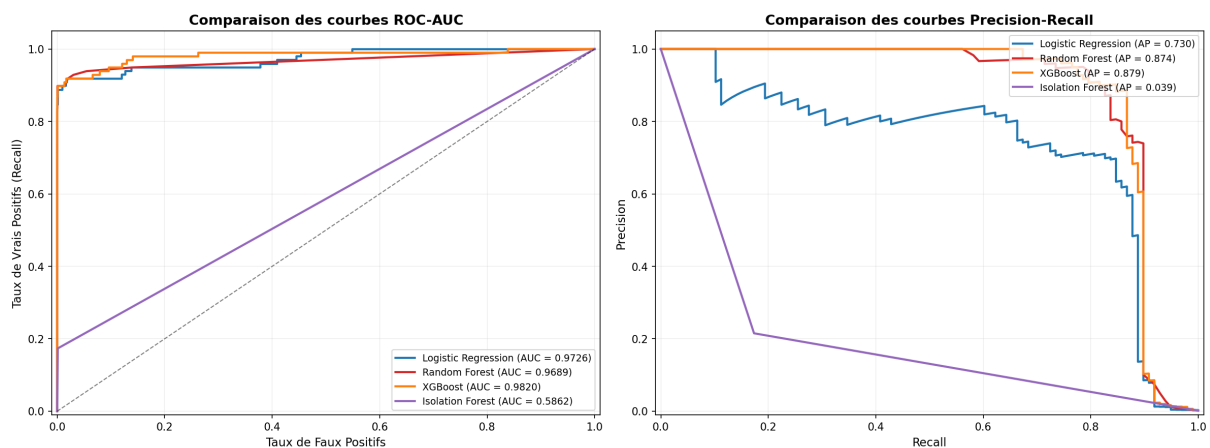


Figure 9. Courbes ROC (gauche) et précision-rappel (droite) des quatre modèles.

6.2 Analyse

XGBoost obtient le meilleur compromis. Il détecte 84 fraudes sur 98 (85,7 % de rappel) et ne signale que 11 transactions légitimes à tort, soit environ 0,02 % d'entre elles ; sur 95 alertes émises, 84 sont justifiées. Son ROC-AUC de 0,9820 dépasse le seuil de 0,95 fixé par le cahier des charges. La Random Forest est très proche ($F_1 = 0,8482$) mais légèrement en retrait sur le rappel.

La **régression logistique** illustre le piège du déséquilibre. Elle a le meilleur rappel (90 fraudes sur 98) mais produit 1 356 faux positifs, soit une précision de 6,2 % : seule une alerte sur seize est justifiée. Par rapport à XGBoost, elle détecte 6 fraudes de plus au prix de 1 345 faux positifs supplémentaires, ce qui bloquerait massivement des clients légitimes. Son accuracy de 97,6 % paraît pourtant correcte, d'où l'importance de ne pas s'y fier. L'écart de *Average Precision* (0,730 contre 0,879 pour XGBoost) reflète mieux cette différence que le ROC-AUC (0,9726 contre 0,9820).

L'**Isolation Forest** reste proche du hasard (ROC-AUC 0,5862, AP 0,039). Ce résultat n'est pas surprenant : les variables V_1 à V_{28} sont déjà issues d'une ACP, et une approche non supervisée n'exploite pas les étiquettes disponibles. Dans ce contexte, elle n'est pas compétitive face aux modèles supervisés.

6.3 Seuil de décision

L'évaluation du tableau 6 est réalisée au seuil de 0,5, enregistré dans les métadonnées du modèle. Dans l'application, le seuil opérationnel appliqué à la décision est abaissé à 0,35 afin de rendre le système plus sensible, ce qui augmente le rappel au prix d'un nombre plus élevé de faux positifs. Les métriques ci-dessus ne décrivent donc pas exactement le comportement de l'application en production ; le choix du seuil devrait être fondé sur le coût respectif d'une fraude manquée et d'un client bloqué (section 7).

7. Limites et perspectives

7.1 Limites

- **Évolution des fraudes (*concept drift*)** : les comportements frauduleux changent ; le modèle, entraîné sur deux jours de transactions, doit être réentraîné périodiquement.
- **Interprétabilité limitée** : les variables V_1 à V_{28} étant anonymisées, il est impossible de relier une décision à une caractéristique métier.
- **Faible effectif de fraudes** : 98 fraudes dans le jeu de test rendent les métriques sensibles ; une fraude de plus ou de moins modifie sensiblement le rappel.
- **Prototype** : l'API ne comporte pas d'authentification et n'est pas durcie pour la production.
- **Seuil** : le seuil opérationnel (0,35) diffère du seuil d'évaluation (0,5) et n'a pas été calibré selon une matrice de coûts.

7.2 Perspectives

- intégrer les valeurs SHAP [9] au pipeline et à l'interface pour expliquer chaque décision (une fonction de tracé existe dans `ml/evaluate.py` mais n'est pas encore branchée) ;
- calibrer le seuil de décision à partir d'une matrice de coûts, et calibrer les probabilités ;
- exploiter le champ `true_label` pour constituer une boucle de rétroaction et réentraîner le modèle ;
- ajouter une authentification (par exemple JWT), des tests unitaires et la conteneurisation (Docker) ;
- évaluer des approches hybrides (autoencodeurs combinés à un classifieur supervisé) et des validations temporelles qui respectent l'ordre chronologique.

8. Conclusion et bilan

Ce projet a permis de construire de bout en bout un système de détection de fraude : analyse d'un jeu de données fortement déséquilibré, comparaison rigoureuse de quatre modèles, sélection de XGBoost (ROC-AUC 0,9820, F1 0,8705, 84 fraudes détectées sur 98 pour 11 faux positifs) et déploiement dans une application Flask avec API REST, journalisation SQLite et tableau de bord d'erreurs. Il illustre en particulier que l'accuracy est trompeuse sur des données déséquilibrées et que le choix du modèle doit se faire sur la précision, le rappel et la courbe précision-rappel.

Le tableau 8 confronte la réalisation au cahier des charges.

Table 8. Bilan de conformité au cahier des charges.

Exigence	Statut
Application Flask 3.x, patron Application Factory	Réalisé
Formulaire de saisie d'une transaction	Réalisé
POST <code>/predict</code> (probabilité de fraude en JSON)	Réalisé
GET <code>/stats</code> et GET <code>/history</code> (50 dernières prédictions)	Réalisé
Tableau de bord administrateur (faux positifs / négatifs)	Réalisé
Analyse exploratoire (distributions, corrélations, déséquilibre)	Réalisé
Normalisation <code>StandardScaler</code> , valeurs manquantes	Réalisé
Rééchantillonnage SMOTE (NearMiss disponible)	Réalisé
Comparaison de quatre modèles	Réalisé
Optimisation par <code>RandomizedSearchCV</code>	Réalisé
Sauvegarde du modèle avec <code>joblib</code>	Réalisé
Journalisation des prédictions dans SQLite	Réalisé
Performance : ROC-AUC > 0,95	Atteint (0,9820)
Interprétabilité par valeurs SHAP	Partiel (fonction présente, non intégrée)

Références

- [1] A. Dal Pozzolo, O. Caelen, R. A. Johnson et G. Bontempi, « Calibrating Probability with Under-sampling for Unbalanced Classification », *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2015.
- [2] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot et G. Bontempi, « Learned lessons in credit card fraud detection from a practitioner perspective », *Expert Systems with Applications*, 41(10), 4915–4928, 2014.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall et W. P. Kegelmeyer, « SMOTE: Synthetic Minority Over-sampling Technique », *Journal of Artificial Intelligence Research*, 16, 321–357, 2002.
- [4] L. Breiman, « Random Forests », *Machine Learning*, 45(1), 5–32, 2001.
- [5] T. Chen et C. Guestrin, « XGBoost: A Scalable Tree Boosting System », *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794, 2016.
- [6] F. T. Liu, K. M. Ting et Z.-H. Zhou, « Isolation Forest », *IEEE International Conference on Data Mining (ICDM)*, 413–422, 2008.
- [7] J. Bergstra et Y. Bengio, « Random Search for Hyper-Parameter Optimization », *Journal of Machine Learning Research*, 13, 281–305, 2012.
- [8] T. Saito et M. Rehmsmeier, « The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets », *PLoS ONE*, 10(3), e0118432, 2015.
- [9] S. M. Lundberg et S.-I. Lee, « A Unified Approach to Interpreting Model Predictions », *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [10] G. Lemaître, F. Nogueira et C. K. Aridas, « Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning », *Journal of Machine Learning Research*, 18(17), 1–5, 2017.
- [11] F. Pedregosa *et al.*, « Scikit-learn: Machine Learning in Python », *Journal of Machine Learning Research*, 12, 2825–2830, 2011.

A. Exemple d'appel de l'API

Requête POST `/predict` au format JSON (les variables V_3 à V_{28} , omises ici, sont à fournir de la même manière) :

```
curl -X POST http://localhost:5000/predict \  
-H "Content-Type: application/json" \  
-d '{"Time": 406, "Amount": 149.62, \  
    "V1": -1.3598, "V2": -0.0728, "V3": 2.5363, ...}'
```

Réponse (exemple) :

```
{  
  "is_fraud": true,  
  "fraud_probability": 0.923,  
  "risk_level": "HIGH",  
  "model_name": "XGBoost",  
  "prediction_id": 1  
}
```

B. Installation et exécution

```
git clone https://github.com/Bilal-Jellaoui/fraud-detection-flask-ml.git  
cd fraud-detection-flask-ml  
python -m venv venv  
venv\Scripts\activate          # Windows  
pip install -r requirements.txt  
python run.py                  # http://localhost:5000
```

Pour réentraîner le modèle, placer `creditcard.csv` (Kaggle) dans le dossier `data/`, puis exécuter `python ml/train.py`.